



# 파이프라인 설명

2025 빅데분 실무 팀프로젝트 모델 구축을 위한 파이프라인 설명 페이지

## 목표

- 온라인 쇼핑몰 구매 데이터를 활용한 추천 시스템 구축

## 구성

- 추천 시스템 = 협업 기반 필터링 + 콘텐츠 기반 필터링

구매 가능성 향상 모델 개발

협업 기반 필터링  
(COLLABORATIVE FILTERING)

나와 비슷한 사용자가 구매한 상품 추천

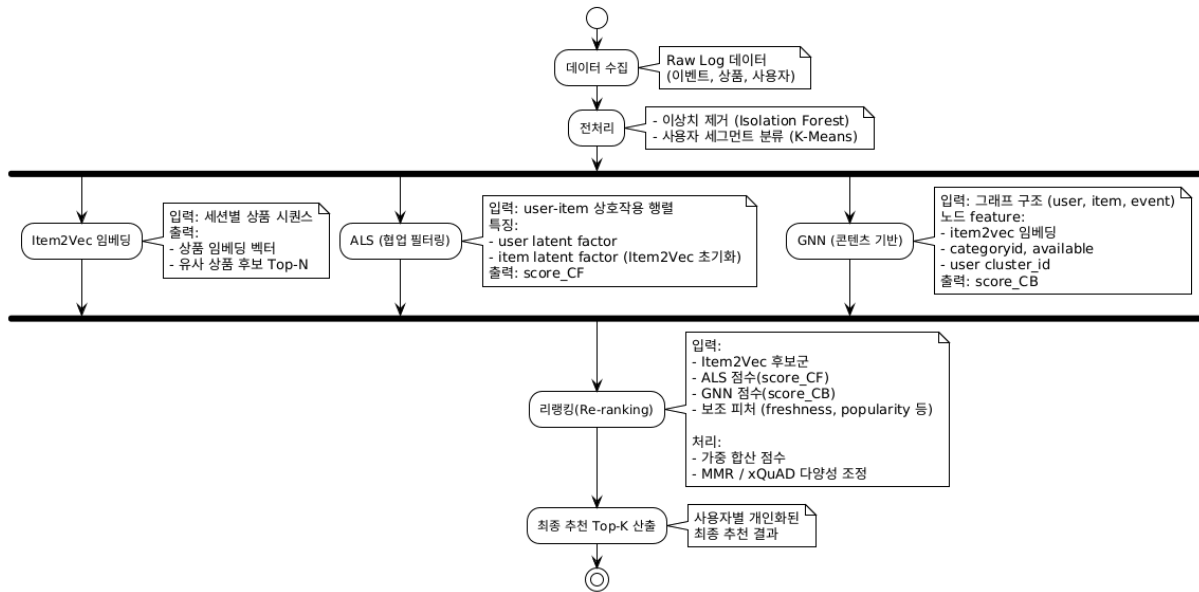
추천 시스템

+

콘텐츠 기반 필터링  
(CONTENT-BASED FILTERING)

내가 구매한 상품과 비슷한 상품 추천

## 추천 시스템 통합 파이프라인 (ALS + GNN + Item2Vec)



데이터 전처리 → {임베딩 + CF + CB} → 리랭킹 → 최종 추천의 구조입니다

## 알고리즘 및 논문 설명

### ▼ 전처리

#### 이건 무슨 과정인가요?

→ 이상 사용자 제거, 사용자 군집화를 합니다

→ 데이터를 전처리해서 뒤의 모델에 넣을 수 있는 형태로 가공하는게 목표입니다

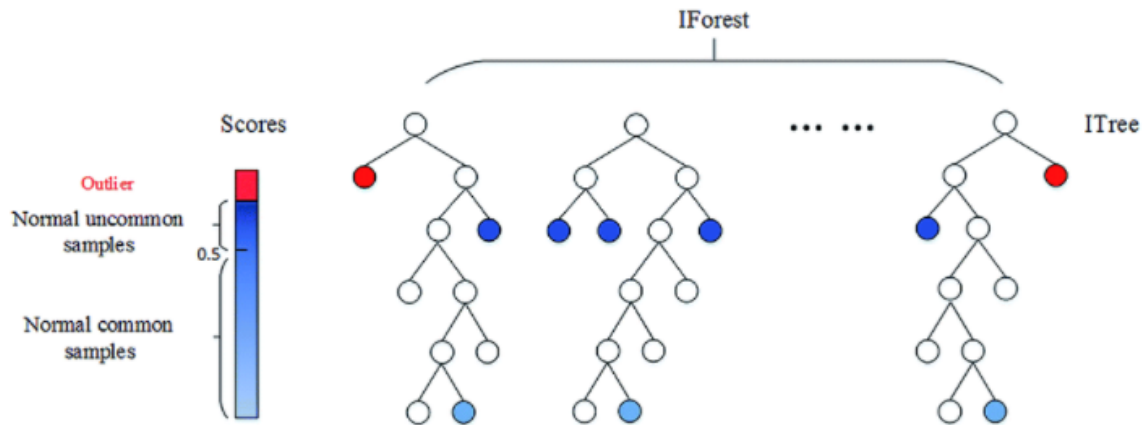
#### 이상 사용자 제거

→ 이후 모델의 학습에서 편향된 결과를 출력하지 않도록 이상 사용자 제거

→ Isolation Forest 사용 예정

#### Isolation Forest

이건 RF의 변종인데, 아래 사진을 보면 이해가 쉬움



저기 빨간색 Outlier가 이상치인데, 직관적으로 비정상적인, 평균적인 데이터와 다른 데이터 라면 RF에서 루트노드에 가깝게 고립되겠지요?

조금 더 자세히 설명하면, IF(Isolation Forest)는 여러 RF의 집합으로 구성됨

이 때 각 RF가 서로 다른 데이터를 처리할 수 있게 전체 데이터에서 일부만 랜덤하게 샘플링 해서 사용함

이후, 각 RF에서는 분기에 사용할 변수  $q$ 를 랜덤하게 선택함

계속 분할하다가 노드에 샘플이 하나만 남거나, 아니면  $\log_2(\text{데이터 개수})$  (트리의 최대 깊이)에 도달하면 멈춤

분류가 끝나면 각 데이터마다 리프 노드까지의 깊이를 기록함 (루트 노드에서 출발해서 리프 노드까지 3번의 분기를 거쳤으면 깊이는 3이라고 기록해놓는 식)

이후 같은 과정을 거친 모든 트리에서 각 데이터에 대해 루트~리프까지의 거리를 구함

그리고 각 데이터에 대해 이 데이터가 포함된 트리들의 깊이의 평균값을 구합니다

예를 들어 어떠한 데이터가 1번 트리에서 3, 4번 트리에서 7, 9번 트리에서 5의 깊이를 가진 다면 해당 데이터의 평균 깊이는 5겠지요

그리고 이 평균값으로 이상치 점수를 규정함 (1에 가까울수록 이상치일 확률 높음)

$$\text{이상치점수} = 2^{-\frac{\text{평균깊이}}{\text{정규화상수}}}$$

그리고 저기 정규화 상수를 어떻게 구하냐면, 각 데이터별로 리프노드에 포함된 데이터가 1 개라면 상수를 1로 하고, 만약 1개보다 크다면 아래와 같이 설정함

$$\text{정규화상수}(1\text{개보다 클 때}) = 2H(n-1) - \left(\frac{2(n-1)}{n}\right)$$

$n$ 은 리프노드에 포함된 데이터 개수이고,  $H(n)$ 은 조화수를 뜻하는데,

$$H(n) = \ln(n) + 0.5772$$

로 근사 가능함

**그런데, 트리로 분리하기에는 feature의 개수가 너무 작은데?**

그래서, 새로운 feature를 만드려 한다

| feature 이름            | 의미             | 계산 방식                      |
|-----------------------|----------------|----------------------------|
| n_views               | 조회 횟수          | event='view' count         |
| n_adds                | 장바구니 추가 횟수     | event='addtocart' count    |
| n_purchases           | 구매 횟수          | event='transaction' count  |
| add_to_view_ratio     | 장바구니/조회 비율     | n_adds / n_views           |
| purchase_to_add_ratio | 구매/장바구니 비율     | n_purchases / n_adds       |
| unique_items          | 방문자가 본 상품 수    | np.unique(itemid)          |
| active_days           | 활동한 날짜 수       | np.unique(date(timestamp)) |
| mean_interval         | 이벤트 간 평균 시간 간격 | 평균( $\Delta$ timestamp)    |
| category_diversity    | 카테고리 다양성       | unique(categoryid) 수       |

위의 표는 예시로, 뭐 이런 느낌으로 feature의 개수를 늘리면 될거같다

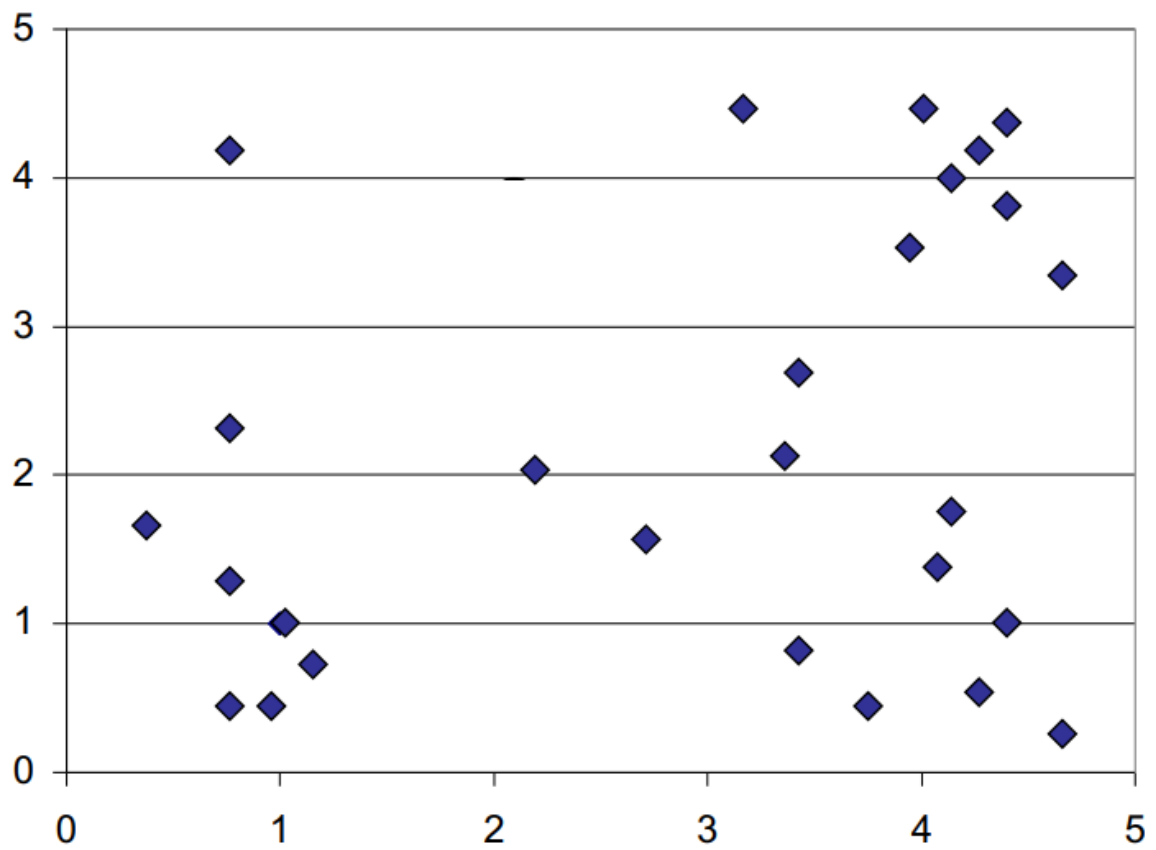
## 사용자 군집화

→ 이상 사용자가 제거된 데이터에서 비슷한 사용자끼리 묶는 느낌

→ K-means를 쓸 예정

## K-means Clustering

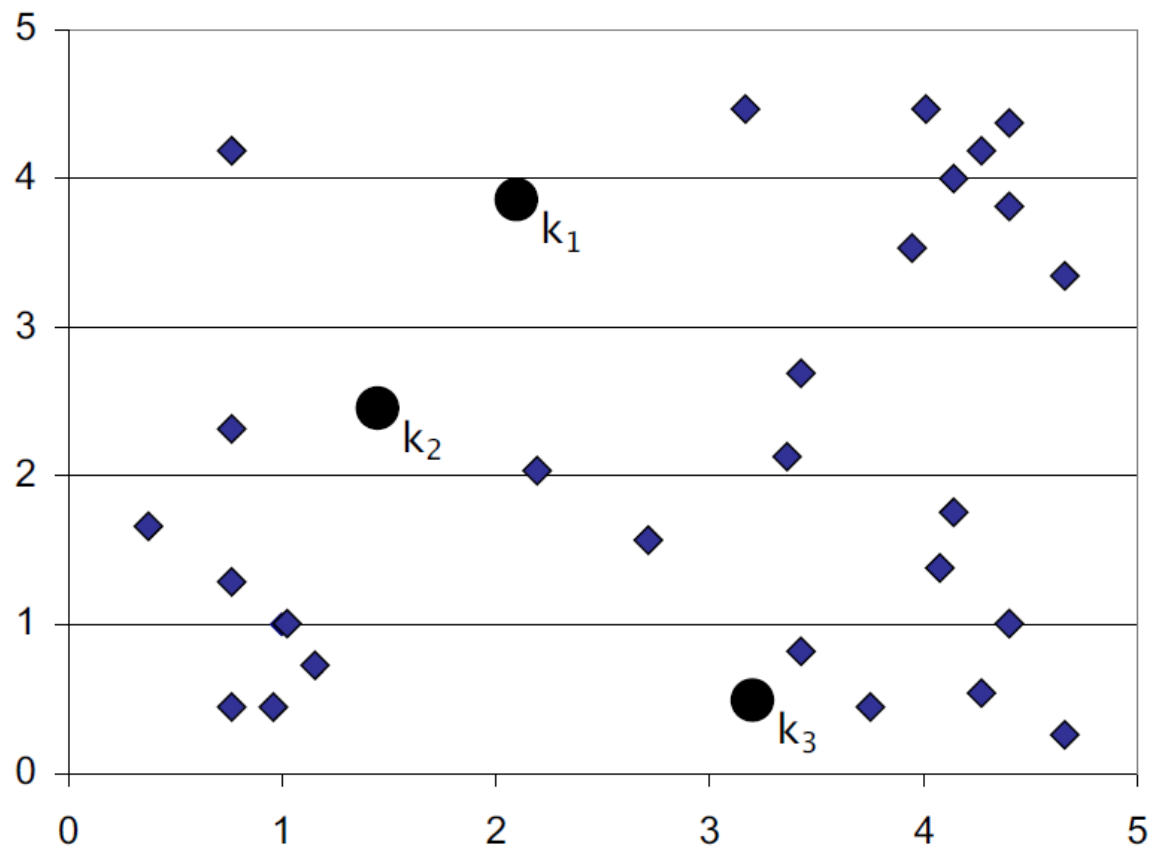
이건 뭐냐면, 비슷한 데이터끼리 묶는 과정이다



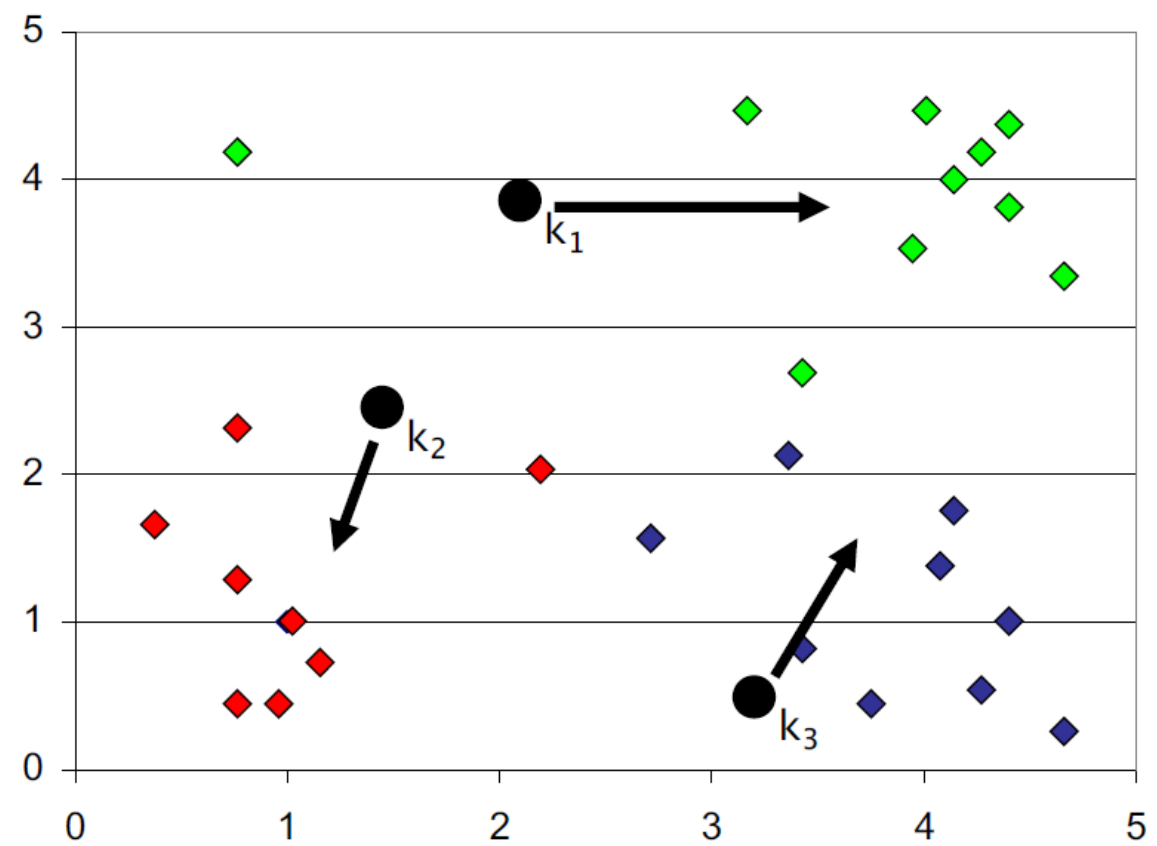
이러한 데이터를 k-means로 군집화하기를 원한다고 해 보자

첫 번째로, 이 데이터들을 몇 개의 묶음으로 나눌 지 묶음의 개수를 정한다

여기서는 3개로 나눈다고 가정하겠음

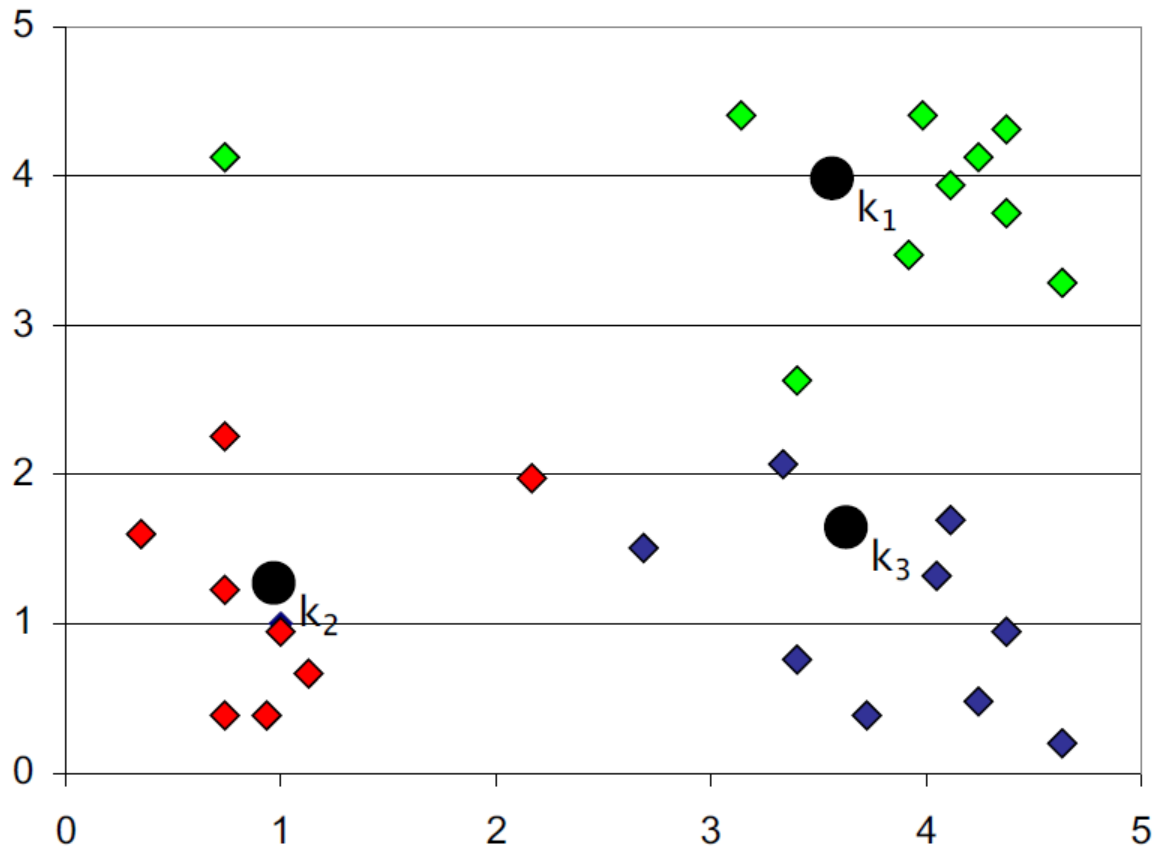


두 번째로, 묶음 개수만큼의 중심점을 아무데나 찍는다



세 번째로, 각 중심점에 가장 가까운 점들은 같은 색으로 칠한다

이후 이 같은 색깔들의 점들의 중심점을 계산하고, 아까 찍었던 중심점을 점들의 중심으로 옮긴다



그럼 이러한 모양이 되는데, 세번째 과정을 반복하면서 더 이상 군집의 요소가 변하지 않을 경우 종료한다

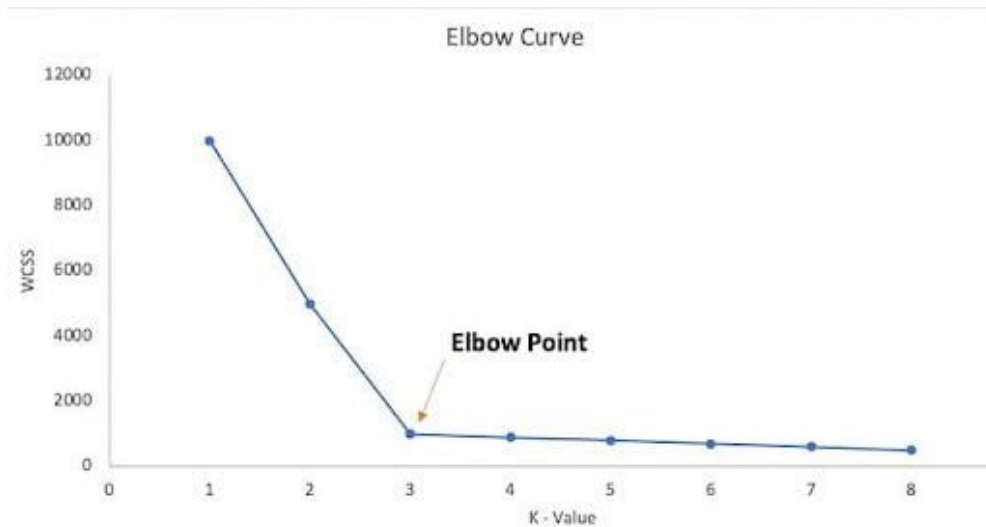
## 그럼 최적의 군집 개수 어떻게 정함?

elbow method라는 방법이 있다

우선 묶음의 개수를 1부터 시작해서 1개씩 늘려가며 군집화를 수행한다

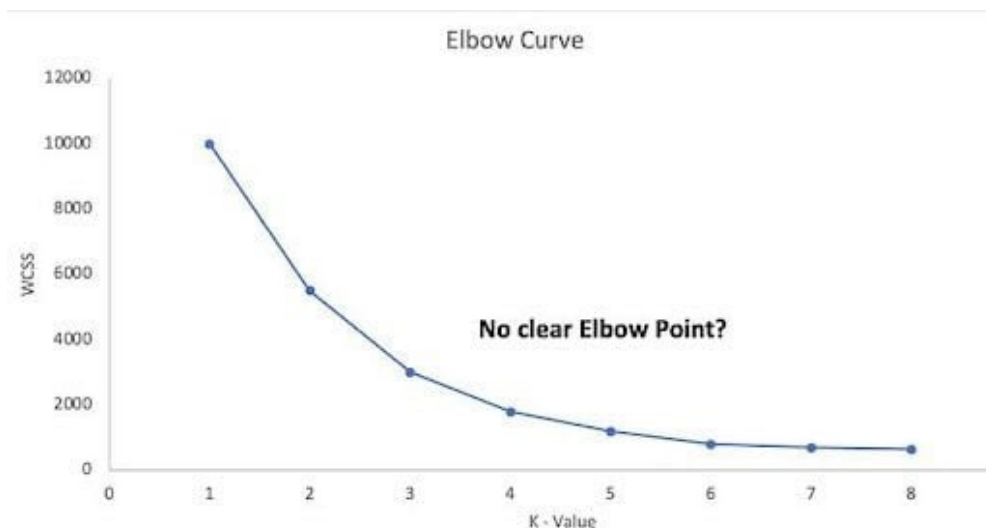
군집화가 완료되었을 때, 군집의 중심점과 각 데이터들간의 거리를 제곱을 취해 곱한다

묶음의 개수를 하나씩 늘려가며 거리를 구하면 아래와 같은 그래프가 나올 것이다



세로축은 제곱해서 더한 거리, 가로축은 묶음의 개수이다

이 때 k=3처럼 팔꿈치같이 꺾이는 부분이 나오는데, 저기가 최적의 묶음 개수이다  
(자신과 다음 포인트의 기울기로 판단한다)



그런데 거리가 이렇게 나와버릴수도 있다

그럴때 사용하는 방법으로 silhouette analysis라는게 있음

$$\frac{b - a}{\max(a, b)}$$

여기서 a는 각 군집 내에서 각 데이터간의 거리의 평균을 뜻하고, b는 한 클러스터 안의 데이터와 그 밖의 군집들간의 거리의 평균을 나타냄

각 데이터에 대해 저걸 계산한다



분모는 a랑 b중에서 큰값 쓰라는 뜻임

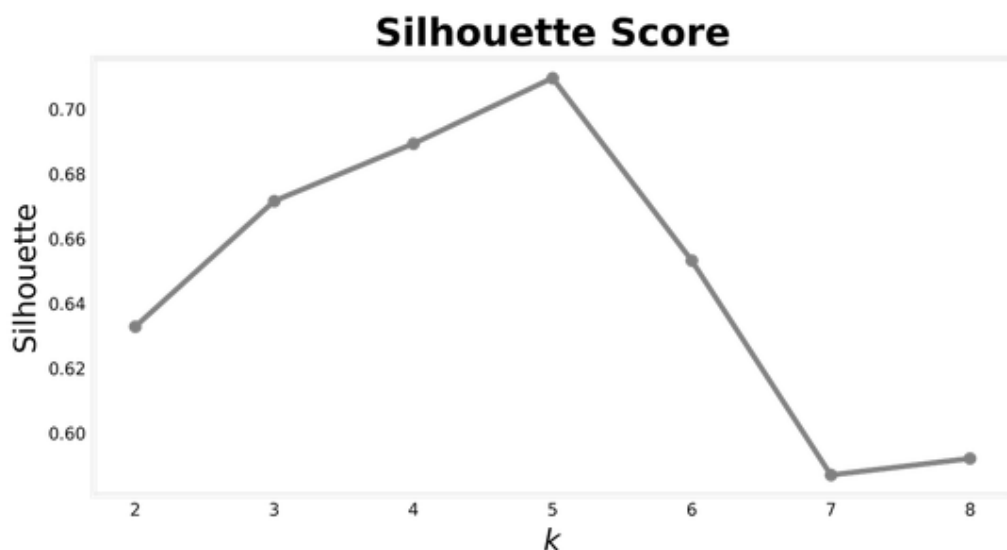
그러니까 a가 작으면 군집의 밀도가 높은거고, b가 높으면 이 군집이 다른 군집과 멀리 떨어져있다는 뜻이다

이 값을 Silhouette Coefficient라 하는데, -1~1의 범위를 가진다

1에 가까우면 다른 군집들과는 멀리 떨어져있으며 군집의 밀도는 높다는 뜻이고, -1에 가까우면 해당 데이터는 다른 군집과 가까우며 같은 군집에서의 거리가 다른 군집까지의 거리보다 더 멀다는걸 뜻한다

0이면 이 데이터는 두 군집의 사이에 걸쳐있다는걸 의미한다

구조상 이거는 묶음의 개수가 2개일때부터 사용 가능하고, 그래프는 아래와 같이 나온다



이건 무슨 뜻이냐면, 묶음 개수가 4일때도 나쁘지 않지만 최적의 묶음 개수는 5개라는걸 의미한다

## ▼ 추천 물품 후보 생성

각 상품의 관계를 벡터값으로 변환하는 과정을 거칠거임

### Item2Vec

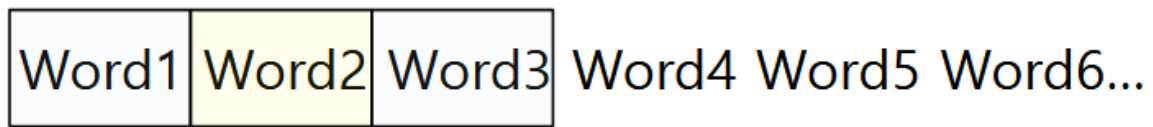
이건 자연어처리의 Word2Vec이라는 알고리즘의 변형이다

먼저 Word2Vec을 간단히 설명하면 아래와 같다

You say goodbye and I say hello 라는 문장이 있다고 하자

You \_\_\_ goodbye and I say hello 이렇게 문장 중간에 구멍을 뚫고, 양 옆의 단어인 You 와 goodbye 를 입력으로 받아서 say 를 출력하는게 Word2Vec 알고리즘이다

이를 위해서 양 옆의 단어를 숫자벡터로 치환하고, 그 두 벡터를 통해서 가운데의 **say** 벡터를 출력한다

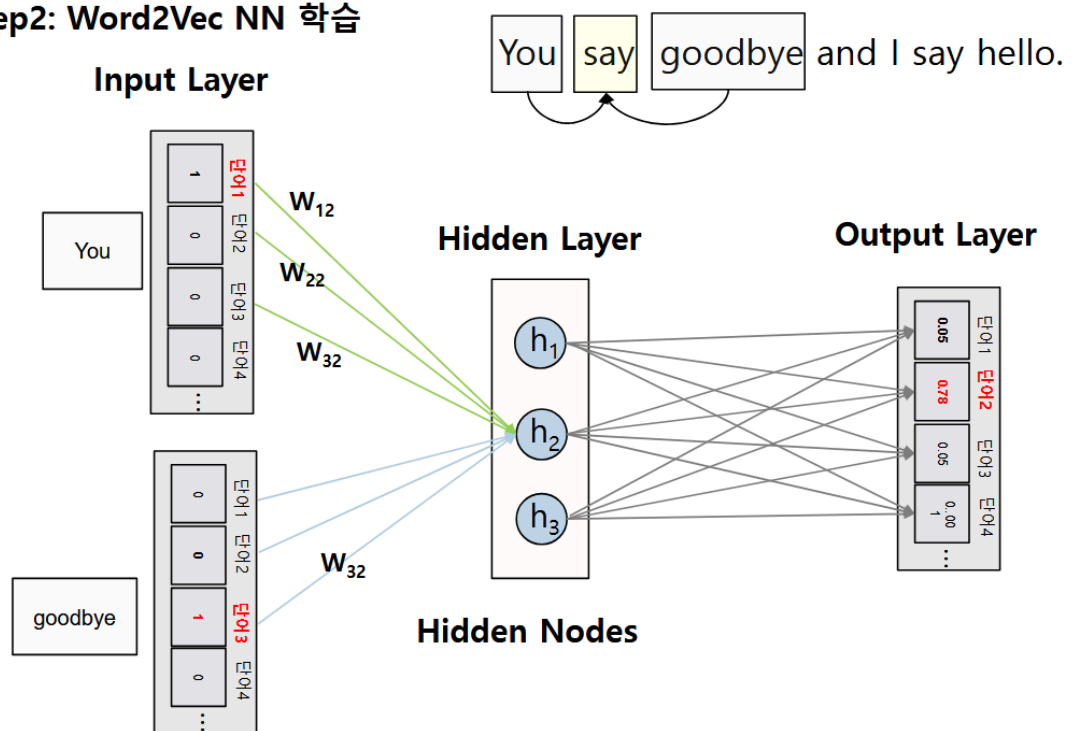


그러니까 이런 구조의 문장에서 Word1과 Word3을 보고 Word2를 추론하는 알고리즘이라고 생각하면 될듯하다

즉, Word2Vec은 주변의 맥락에 대한 정보를 받아서 각 단어의 출현 확률을 출력하는 신경망이다

#### ▼ Word2Vec에 대한 조금 더 자세한 설명

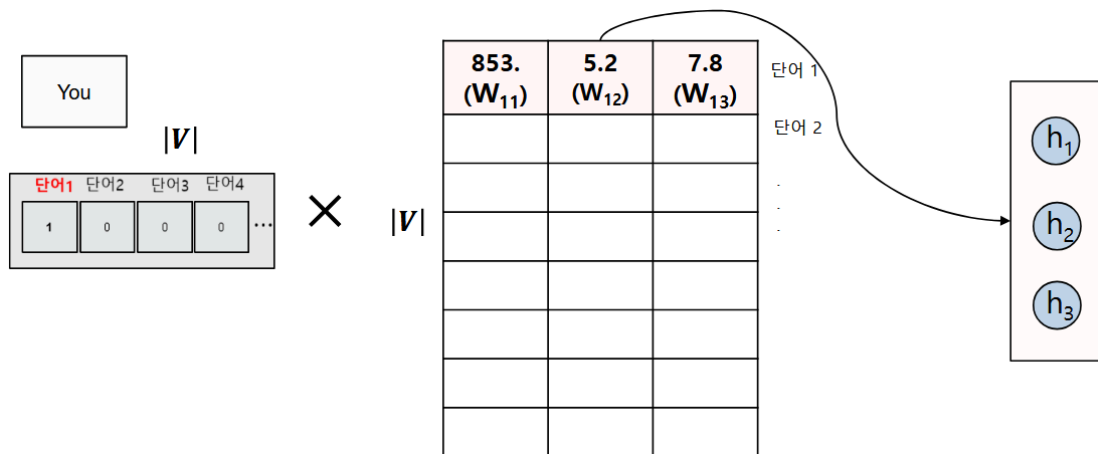
##### Step2: Word2Vec NN 학습



You가 단어 1, say가 단어 2..... 이렇게 구성된 단어벡터가 있다고 해 보자

여기서 입력되는 단어 (사진에서는 You와 goodbye)에 대해 무작위 weight로 신경망을 수행하면, 출력 레이어에는 단어벡터의 모든 값에 대해 값이 짝 나올것이다

간단하게 생각하면 클래스가 단어벡터의 크기만큼 있는 classification 신경망이라고 할 수 있는데, 위 그림에서는 단어 2, 즉 say의 확률이 가장 높기 때문에 You와 goodbye의 사이에 올수 있는 단어 중 가장 높은 확률이 say라는것을 알 수 있다



이렇게 하면 각 단어벡터에 따라 Input~Hidden 레이어 사이의 weight에 대한 벡터가 정해지게 된다

이때 각 단어에 따라 weight가 달라지게 된다

자세히 말하자면, 저 큰 행렬이 weight로 사용되는건 동일한데, 첫 번째 단어는 weight의 첫 번째 행을 weight로 쓰고, 세번째는 세번째 행을 weight로 쓰고 이런 식이다

그리고 목표인 두번째 단어를 구할때는 첫번째 단어의 weight랑 세번째 단어의 weight를 평균내서 두번째 단어의 weight로 한다

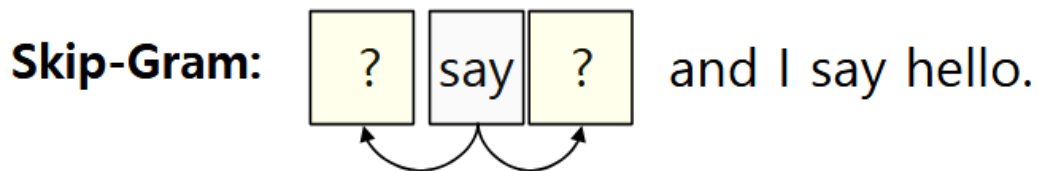
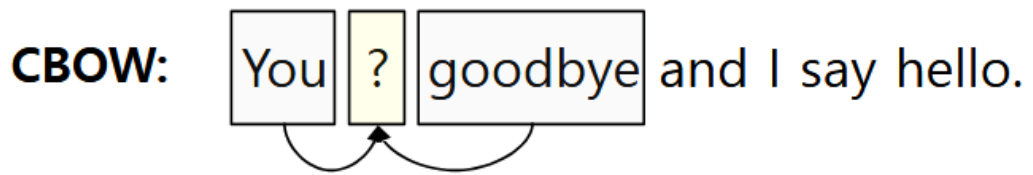
하나의 단어당 하나의 행에만 연결되기 때문에, 사실상 하나의 행이 한 단어를 숫자로 요약해놓은거랑 같은 의미가 된다

Item2Vec은 이 Word2Vec의 아이템 버전이다

Word2Vec은 단어가 문장 내에서 등장하는 패턴을 학습해 단어의 의미를 학습하듯이, Item2Vec은 각 상품들이 사용자들의 세션 안에서 상호작용하는 패턴을 학습해서 상품간의 관계를 학습하는것이 목표이다

이를 위해서 Skip-Gram을 사용하려 한다

다시 Word2Vec으로 돌아가보면, 이런 느낌이다



CBOW는 기존의 Word2Vec에서 쓰이는 방식이고, Skip-Gram은 그걸 뒤집은 방식인데 더 어려운 대신 높은 정확도를 기대할 수 있다

Word2Vec에서는 단어들로 구성된 문장에 대해 가운데의 단어(CBOW) 혹은 양 옆의 단어(Skip-Gram)을 학습하지만, Item2Vec에서는 단어는 아이템의 ID, 문장은 세션으로 치환하면 될거같음

다시한번 목표를 정리하자면 아래와 같다

“아이템 A가 나타났을 때 주변에 아이템 B가 등장할 확률이 높게 되도록 하기”

## 어떻게 하나요

1. 우선 세션을 만들어야 한다  
같은 visitorid의 이벤트를 시간 순으로 정렬해서 30분마다 하나의 세션으로 구분한다고 가정하자
2. 각 세션에 속하는 아이템에 대해, 이 세션을 itemid 벡터로 변환한다  
그러니까 한 세션 안에 302, 541, 879번 아이템에 대해 상호작용이 있었다면 이 세션은 1번세션 이런 이름이 아니라 [302, 541, 879]로 부른다는 소리다
3. 이제 이 세션(itemid 벡터) 내에서 위의 skip-gram 방식으로 학습한다

이렇게 하면 각 상품들이 벡터로 요약되게 된다

이 벡터들에 대해 코사인 유사도를 통해 각 아이템에 대해서 유사도가 높은 아이템을 뽑을 수 있게 된다

다만 GNN과는 다른것이, Item2Vec은 한 세션 내의 물품들에 대해서만 유사도를 학습할 수 있다

예를 들어서 아래의 두 세션이 있다고 가정해보자

- 1번 세션: 노트북, 마우스, 키보드
- 2번 세션: 노트북, 가방, 충전기

이걸 item2vec이 학습하면 노트북 ↔ 키보드, 노트북 ↔ 가방이 유사하다고 판단할 수 있음

그런데 키보드 ↔ 가방은 전혀 관련이 없다는 점을 모름 (다른 세션에 있으니까)

## ▼ CF

### ALS

협업 필터링이란 나와 비슷한 사용자가 구매한 상품을 추천하는 알고리즘임

이를 가장 쉽게 구현하기 위해서, 추천 시스템을 사용자와 상품의 관계를 나타내는 거대한 표를 만드는 것이라고 생각해보자.

| 사용자 \ 구매 여부 | 휴지 | 가위 | 연필 | ..... |
|-------------|----|----|----|-------|
| 민수          | O  |    |    |       |
| 경희          |    |    |    |       |
| 철우          |    |    | O  |       |
| 태범          |    | O  |    |       |
| .....       |    |    |    |       |

위와 같은 식으로 거대한 표를 채우는 거라고 생각하면 된다.

그런데 상품의 종류도 많고, 사용자도 많으니 자연스럽게 표의 대부분은 비어있지 않을까?

→ 이러한 데이터를 희소행렬, Sparse하다고 표현한다.

CF는 기본적으로 이 비어있는 칸을 예측해서 민수가 연필을 살지 말지, 태범이 휴지를 살지 말지 예측하는 문제를 푸는 알고리즘이다.

(이를 수학적으로 말하면 Low-Rank Matrix Reconstruction이라 하고, 링크된 논문의 제목도 이러하다)

이제 CF가 풀어야 하는 문제를 알았으니, 다음으로 ALS가 뭐하는 알고리즘인지 살펴보자

ALS는 이 문제를 해결하기 위해 우선 위의 큰 데이터를 두 조각으로 나눈다

사용자 요인 행렬(L)과 상품 요인 행렬(R)이렇게 두 가지로 나누는데,

- L: 각 사용자가 어떤 성향을 가지고 있는가?
- R: 각 상품이 어떠한 특징을 가지고 있는가?

를 의미한다.

또한, 전체 평점(또는 구매 여부) 데이터는  $L \times R$ 이라는 행렬곱을 통해 근사할 수 있다 (정답을 구하는 것이 아니라 정답을 근사하는 것이고, 정확한 값은 아닌데 애초에 예측하는 알고리즘이라 그렇다)

그런데 중요한 건 저 L, R 두 행렬 모두 직접 구할 수 없다는 것이다

그래서 ALS가 어떻게 동작하냐면

1. L을 임의로 정함 (= 빈 행렬에 랜덤한 값을 채워넣음)
2. 이 L을 이용해서 R을 최소제곱법으로 계산함
3. 이번에는 계산된 R로 L을 최소제곱법으로 계산함
4. 2번과 3번을 반복하며 loss 최소화하기

이렇게 동작한다.

#### ▼ 최소제곱법이 뭔가요

독립변수 X와 종속변수 Y가 있을 때, 이 둘이 선형관계라고 가정한다면 식은 아래와 같이 세워진다.

$$Y = f(X) = aX + b$$

이때 error(=residual)은 아래와 같다

$$e_i = |Y_i - f(X_i)|$$

데이터의 개수만큼 error가 있을텐데, 이 error들을 제곱해서 모두 더한걸 SSE라 한다

$$SSE = \sum_{i=1}^n (Y_i - aX_i - b)^2$$

이후 이 SSE를 a와 b로 편미분한다

$$\frac{\partial(SSE)}{\partial a} = 0, \frac{\partial(SSE)}{\partial b} = 0$$

Y, A, X를 행렬로 표현하면

$$\mathbf{Y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_3 \end{bmatrix}, \mathbf{A} = \begin{bmatrix} x_1 & 1 \\ x_2 & 1 \\ \vdots & \vdots \\ x_3 & 1 \end{bmatrix}, \mathbf{X} = \begin{bmatrix} a \\ b \end{bmatrix}$$

$$\mathbf{A}^T \mathbf{A} \mathbf{X} = \mathbf{A}^T \mathbf{Y}$$

이걸 풀면

$$\mathbf{X} = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{Y}$$

따라서, 최소제곱법이란 변수들이 선형관계인 식에서 파라미터를 구하는 통계적 방법이다

OLS라고도 함

## ALS 예시

회식 메뉴를 정해야 하는데, 사람 100명에 메뉴는 100개중 하나를 정해야 하고, 어떤 사람이 어떤 메뉴를 좋아하는지는 약 10명정도만 알 수 있다고 하자

ALS의 동작방식은 아래와 같다

1. 사람들의 메뉴 취향 (L 행렬 이라고 할 수 있다)을 임의로 추정  
→ 예를 들자면 10명이 동일하게 좋아하는 메뉴가 있다고 해 보자  
그 메뉴가 매운 메뉴일 때, 메뉴 취향은 매운맛을 좋아한다고 추정할 수 있다
2. 그걸 바탕으로 메뉴들의 특징 (R 행렬 이라고 할 수 있다)을 계산  
→ 사람들이 매운맛을 좋아하므로, 이 메뉴는 맵다라고 추정할 수 있다
3. 그 정보를 이용해서 사람들의 메뉴 취향을 보정  
→ 매운걸 좋아하는 사람이라면 이 메뉴에도 높은 점수를 줄 것이다 라고 예측한다
4. 이걸 반복해서 점점 더 정확한 예측을 함

## ALS의 장점

- 사용자 하나당 계산하는 과정이 서로 독립적임, 병렬화로 속도개선 용이
- 수학적으로 최적화된 알고리즘이라 딥러닝같이 확률 기반으로 추론하는 부분 없이 바로 진동 없이 일정하게 수렴하는걸 기대할 수 있음
- 논문에 따르면 데이터에 약간의 노이즈가 있어도 수렴이 가능하며, 이론적 상한치에 근접한다고 설명함

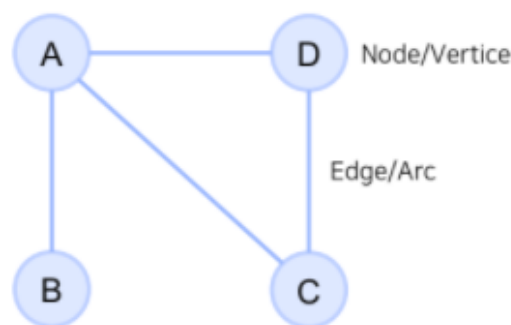
- 해석이 쉬운데, 예를 들어 영화를 추천하는 문제에서 이 사람은 액션영화를 좋아하는데, 이 영화는 장면의 격렬함 지수가 높다. 그래서 추천할만 하다 라고 해석할 수 있고, 추천 이유를 설명하기도 쉽다

## ▼ CB

### GNN

GNN은 그래프 형태의 데이터에 인공지능을 적용하는 신경망 구조이다

#### 그래프가 뭔가요



그래프란 노드와 그 노드들을 잇는 엣지로 구성된 데이터를 의미한다

엣지는 방향과 가중치를 가질 수 있다

#### 이거 왜 쓰나요

csv처럼 표로 정리하기 쉬운 데이터만 있으면 참 좋겠지만, 세상에는 그렇지 않은 데이터들도 많다.

SNS의 인간관계, 상품 간의 유사성 데이터 등등은 표로 정리하기 참 힘든 데이터들이다

#### 어떻게 동작하나요

이름에 신경망 (Graph **Neural Network**)가 들어간 만큼 자세하게 설명하려면 신경망부터 먼저 설명해야 하지만, 최대한 쉽게 설명하자면 GNN이란 각 노드가 옆에 있는 노드에게서 정보를 받아서 노드 자기 자신을 업데이트하는 구조이다

이렇게 각 노드가 이웃에게서 정보를 받아 자기 상태를 업데이트하는 구조를 message passing / information aggregation이라고 한다

예를 들어서 영화를 추천하는 시스템이라면 노드로는 사용자나 영화가, 엣지로는 "이 사용자가 영화를 봤다"라는 정보가 될 수 있다.

그리고 GNN은

1. 사용자 노드는 자신이 본 영화들의 정보를 수집한다



2. 영화 노드는 자신을 본 사용자들의 정보를 수집한다
3. 이 정보들을 주고받으며 서로의 표현 (나는 어떤 사람이고, 나는 어떤 영화다 이거)을 점점 더 정밀하고 자세하게, 깔끔하게 만든다

의 과정을 거쳐 동작하게 된다

## GNN 설계방법

1. 그래프 구조 찾기
  - a. 데이터가 원래 그래프 형태 → 그대로 사용
  - b. 아니라면 그래프 형태로 데이터 수정
2. 그래프 유형 정의
  - a. 엣지의 방향이 있는가?
  - b. 노드와 엣지가 여러종류인가?
  - c. 시간에 따라 그래프의 형태가 변하는가?
  - d. 위의 것들을 종합하여 그래프 유형 정의
3. 목표 정의
  - a. 노드, 엣지, 그래프 전체 중 뭘 예측할지
  - b. 예를 들어 노드 분류면 사용자 성별 예측, 엣지 예측이면 누가 누구를 팔로우할까? 같은 것
4. 모듈 설계 및 조립
  - a. Propagation, Sampling, Pooling 등 모델 설계 및 조립

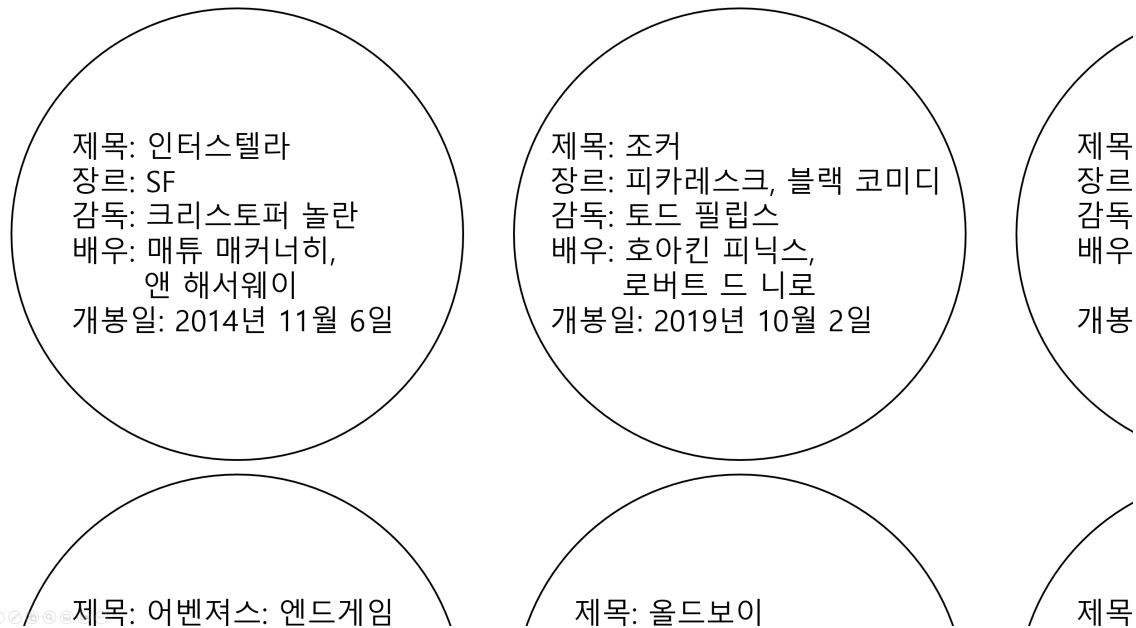
## 그래서 이거 CB에 어떻게 씬

전통적인 CB 알고리즘은 각 콘텐츠를 벡터로 표시하고, 코사인 유사도 등의 벡터간의 유사도를 측정하는 기법으로 추천할 데이터를 선정한다

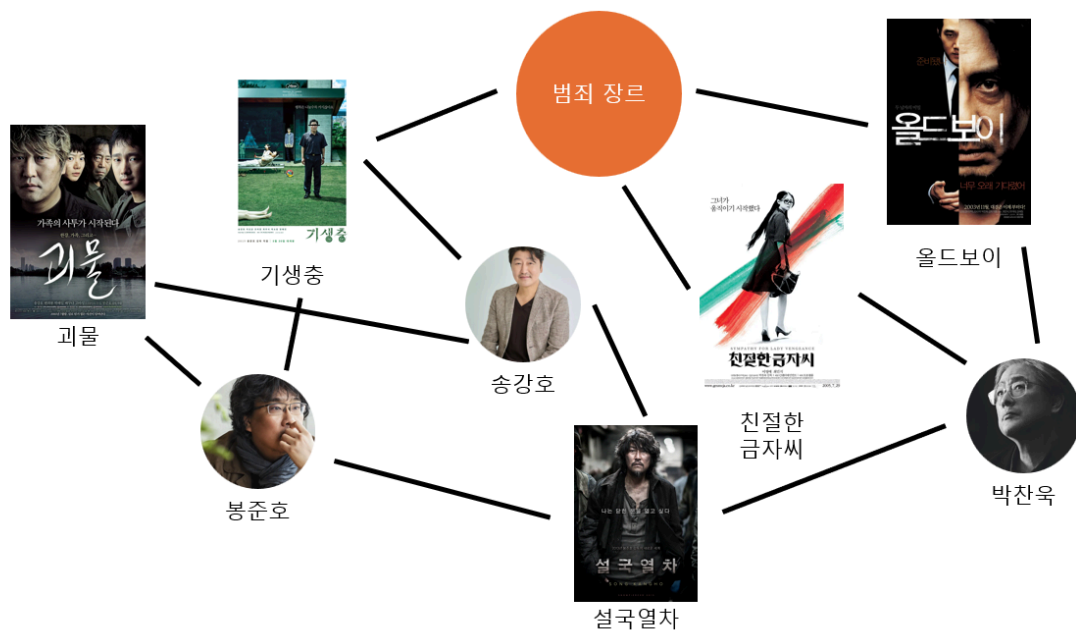
예를 들어 영화 추천이라면 영화의 장르, 배우 및 감독, 영화의 키워드 등이 이 영화의 특징이 되어 벡터로 표현되는 느낌임

근데 이 방식에는 약점이 있다

1. 이 배우가 저 영화에도 나왔다같은 연결을 반영하기 어려움
2. 감독 + 배우의 조합과 같은 특징 사이의 상호작용이 무시됨
3. 이 영화와 비슷한 주제의 웹툰처럼 새로운 특징이 생기고 다층적 관계가 생기면 기존의 방식에 적용하기 어려움



기존의 벡터 방식은 이렇게 영화에 대한 정보를 저장해놓고, 다른 벡터들과의 유사도를 측정하는 방식이다



반면 그래프 방식은 이렇게 영화, 감독, 배우, 장르 등의 특징들을 노드로 해서 각 노드 사이의 연결관계를 표현하는 방식이다

위의 한국영화 그래프에 GNN을 적용하게 되면 각 노드들이 이웃한 노드로부터 정보를 받아 자기 자신을 업데이트한다.

이 때 확률을 기반으로 업데이트되는 신경망의 특성에 따라 같은 작가의 작품이지만 장르가 다르다면 관계를 약하게, 같은 장르라도 자주 함께 소비된다면 연결고리를 강하게 하는 식으로 각 노드들 사이의 문맥 또한 학습된다

이렇게 하면 학습이 완료된 그래프에서 사용자에게 어떠한 노드를 추천해줄지를 연결고리의 강도를 보고 쉽게 파악할 수 있다

예를 들어, 위의 그래프에서 사용자가 [괴물]을 보고 좋은 평가를 남겼다면 같은 감독의 작품인 [기생충]이나 [설국열차]를 추천해주는 식이고, [기생충]을 보고 좋은 평가를 남겼다면 같은 장르인 [올드보이]나 [친절한 금자씨]를 추천해주는 식이다

이는 각 사용자마다 독립적으로 반영할 수 있어 사용자에게 비슷한 장르의 영화와 같은 배우가 나오는 영화 이렇게 두 개를 추천해줬는데 그 중 비슷한 장르의 영화를 골랐을 경우, 신경망은 다음부터는 같은 배우보다는 비슷한 장르의 영화를 더 추천하는 방향으로 학습된다

## ▼ 리랭킹

여기서는 Item2Vec으로 나온 임베딩 벡터, ALS, GNN의 결과를 섞어서 최종 추천 아이템을 선별하는게 목표임

- ALS: 유사한 사용자들이 선호한 물건 추천
- GNN: 이 아이템과 비슷한 물건 추천
- Item2Vec: 이 세션 내에서 서로 비슷한 물건 추천

이 결과들에 대해서 아래와 같은 방식으로 최종 추천 점수를 산출할 계획임

- 각 모델들을 정규화한 후, 가중치를 곱해서 더함 (예: ALS는 0.45, GNN은 0.25 뭐 이런식)
  - 이 가중치는 나중에 GridSearch로 조정하면 될듯
- MMR (Maximal Marginal Relevance) 적용
  - 근데 이럼 비슷한 제품만 잔뜩 추천하는 문제가 있을 수 있다  
이걸 조금이나마 완화하기 위해 MMR을 쓸거임
  - MMR은 아래와 같이 구한다

$$MMR = \lambda \times \text{가중합} - (1 - \lambda) \times \max(\cos(\text{두 아이템}))$$

람다는 0.6~0.8 사이정도에서 조정하면 될거같음

- 재고 있는지 검사
  - available이 1인 제품들에 추가 가중치를 곱한다  
1.5라던가 이렇게 재고가 있는 상품을 더 추천하게 하면 될듯하다
- 이렇게 해서 최종적인 추천 리스트가 만들어지게 된다